

Foundations of Supervised Learning

Motivation: Why Supervised Learning?

- Many real-world problems require predicting unknown outcomes.
- Examples:
 - Email spam detection
 - Loan approval
 - Medical diagnosis
 - Student performance prediction
- Idea: learn patterns from historical labeled data.

Supervised Learning Techniques

What will we learn in this module?

We will establish the fundamental concepts required to understand all supervised learning algorithms.

- **Defining** Supervised Learning.
- **Distinguishing** between the two main pillars: Classification and Prediction (Regression).
- Understanding the **high-level objectives** and mathematical goals.
- Exploring **real-world application scenarios**.

Learning Goals for This Module

The "Teacher" Analogy

Imagine a teacher showing a student pictures of fruits (apples and oranges) and telling them which is which.

- **Training Phase (Learning):** The algorithm is given a dataset containing **input features** and the **correct output (label)** . The algorithm learns a function that maps the input to the output.
- **Testing Phase (Prediction):** The algorithm is given new, unseen data. It applies the learned function to predict the output.

- **Formal Definition:**

Given a set of input variables **X** and an output variable **Y**, the goal is to learn a mapping function **f: X -> Y**.

- **X** = Features / Predictors / Independent Variables
- **Y** = Target / Label / Response / Dependent Variabel

Basic Idea of Supervised Learning

A model learns mapping from inputs to outputs.

Inputs $\rightarrow$ Features (X)

Outputs $\rightarrow$ Labels/Target (Y)

- Goal: learn function

$$f(X) \rightarrow Y$$

- that predicts correct output for unseen data.

Training Dataset Representation

- Dataset representation:
 - $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$
- Where
 - x_i = feature vector
 - y_i = target value
- Example feature vector:
 - $x = (x_1, x_2, x_3, \dots, x_d)$
- d = number of features.

The Two Pillars: Classification vs. Prediction

The nature of the output variable Y determines the type of problem.

1. Classification (Categorical Output)

- Y is a category or a class label.
- **The algorithm predicts a discrete class.**
- *Example:* "Is this email Spam or Not Spam?" ($Y = \{\text{Spam}, \text{Not Spam}\}$)

2. Prediction / Regression (Continuous Output)

- Y is a real number or a continuous value.
- **The algorithm predicts a quantity.**
- *Example:* "What will be the selling price of this house?" ($Y = \$ \text{ price}$)

Key Differences: A Side-by-Side Comparison

Feature	Classification	Prediction (Regression)
Output Type	Discrete / Categorical	Continuous / Numerical
Goal	To assign data to a class	To forecast a numerical value
Evaluation	Accuracy, Precision, Recall, F1-Score	Mean Squared Error (MSE), R-squared
Example Task	Diagnose patient: (Disease A, B, or C)	Predict patient's blood pressure reading
Visualization	Decision Boundary	Line/Curve of Best Fit

FEATURES, LOSS FUNCTIONS AND TRAINING

Feature Space

- Features describe each object.
- Example student features:
 - Hours studied
 - Attendance
 - Assignments submitted
 - Previous GPA
- Feature vector representation:
 - $x = (x_1, x_2, x_3, x_4)$
- Each x_i represents one attribute.

Target Variable

The variable the model must predict.

Two types exist:

- Categorical → Classification
- Numeric → Prediction (Regression)
- Examples:
 - Pass / Fail
 - Spam / Not spam
 - Exam score
 - House price

Hypothesis Function

- The model learns a hypothesis function:
 - $h(x)$ = predicted output
- Examples of hypothesis models:
 - Decision trees
 - Naïve Bayes classifiers
 - Neural networks
 - Support Vector Machines

Loss Function

- Loss function measures prediction error.
- Example:
 - $\text{Error} = \text{Actual} - \text{Predicted}$
- Common loss measures:
 - Squared error
 - Absolute error
 - Classification error
- Learning objective:
 - Minimize total prediction error.

Types of Supervised Learning Tasks

- Two major categories exist:
- Classification
- Prediction (Regression)
- Both attempt to learn $f(X) \rightarrow Y$ but differ in output type.

Classification Tasks

- Output variable is categorical.
- Examples:
 - Spam vs Not Spam
 - Disease vs No Disease
 - Pass vs Fail
 - Fraud vs Legitimate
- Mathematical form:
 - $f(x) \in \{C_1, C_2, \dots, C_k\}$

Prediction (Regression) Tasks

- Output variable is numeric.
- Examples:
 - House price prediction
 - Temperature prediction
 - Stock price forecasting
 - Exam score prediction
- Mathematical form:
 - $f(x) \in \text{Real numbers}$

Classification vs Prediction

- Classification
 - Output categorical
 - Finite label set
- Prediction
 - Output numeric
 - Continuous values
- Examples:
 - Classification → email spam detection
 - Prediction → house price estimation

Mathematical Model of Data

- Real-world data often follows:
 - $y = f^*(x) + \text{noise}$
- Where
 - $f^*(x)$ = true unknown function
 - noise = random variability
- Goal of learning algorithm:
 - estimate $f(x) \approx f^*(x)$

Error Minimization Objective

- Learning algorithm tries to minimize prediction error.
- Example loss:
 - Squared error = $(y - f(x))^2$
- Total training error:
 - Sum of squared errors over all samples.

The Mathematical Objective

The core goal of any supervised learning algorithm is to find a function $\mathbf{f}(\mathbf{X})$ that approximates the true relationship between inputs and outputs.

The Model:

$$Y = f(X) + \epsilon$$

- $\mathbf{f}(\mathbf{X})$: The systematic information we try to learn from X .
- ϵ (**epsilon**): The random error term, irreducible noise. This is the part we cannot predict.

The Goal of Learning:

- To find the function $\hat{\mathbf{f}}$ (**f-hat**) that best estimates $\mathbf{f}$. We want $\hat{\mathbf{f}}$ to be as close as possible to $\mathbf{f}$ for all possible inputs, minimizing the error.

Objective: Minimizing Error

To find the best $\hat{f}$, we need a way to measure how wrong our predictions are. This is done with a **Loss Function**.

- **A. Loss Function for Classification (0-1 Loss):**
 - We count a mistake as 1, a correct prediction as 0.
 - We want to minimize the total number of misclassifications.
 - *Example:* If the true label is "Spam" and we predict "Not Spam", Loss = 1.
- **B. Loss Function for Prediction (Squared Error Loss):**
 - Measures the squared difference between the actual value (y) and the predicted value ($\hat{f}(x)$).
 - **Formula:**
 - $L(y, \hat{f}(x)) = (y - \hat{f}(x))^2$
 - We square the error to penalize large errors more heavily and to ensure the error is always positive.

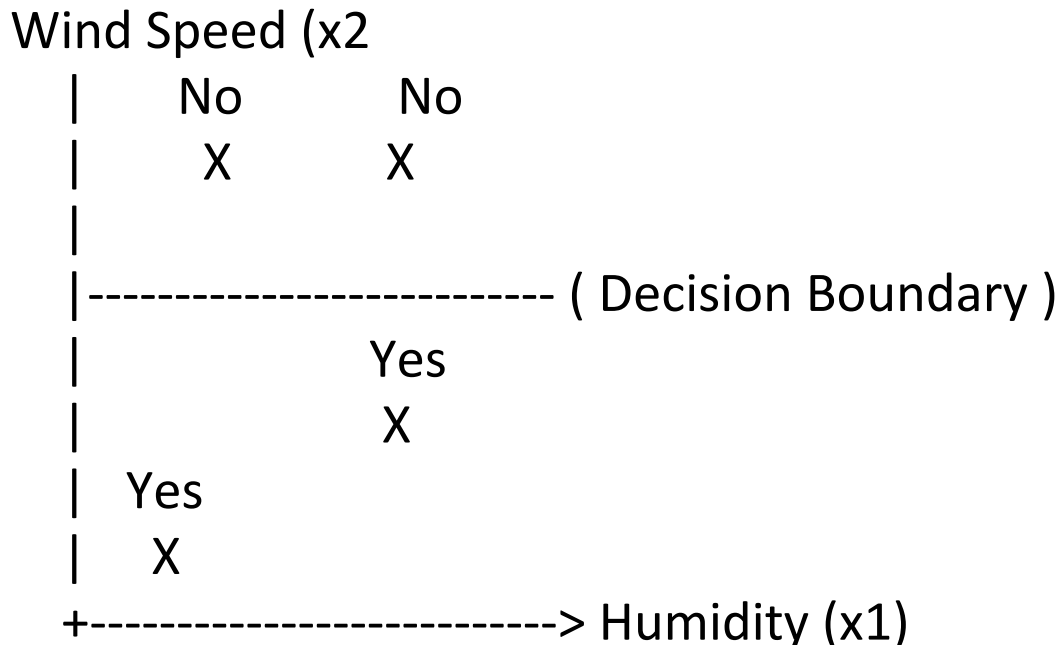
A Toy Example for Classification

- **Problem:** Predict whether a student will **Play Tennis** (Yes/No) based on weather conditions.
- **Toy Training Data:**
 - **Input Features (X):** Outlook, Humidity, Wind.
 - **Output (Y):** Play Tennis (Yes or No) -> **Classification Problem**

Outlook	Humidity	Wind	Play Tennis? (Y)
Sunny	High	Weak	No
Sunny	High	Strong	No
Overcast	High	Weak	Yes
Rainy	Normal	Weak	Yes
Rainy	Normal	Strong	No
Overcast	Normal	Strong	Yes

Visualizing a Simple Classifier (Decision Boundary)

- Let's simplify the problem to two numerical features for visualization: **Humidity (x1)** and **Wind Speed (x2)**.
- Goal:** Draw a line (or curve) that separates the "Yes" class from the "No" class. This line is called a **Decision Boundary** for **playing Tennis Problem**



- Classification Rule:** Any new data point falling on one side of the boundary is classified as "Yes", and on the other side as "No".

A Toy Example for Prediction (Regression)

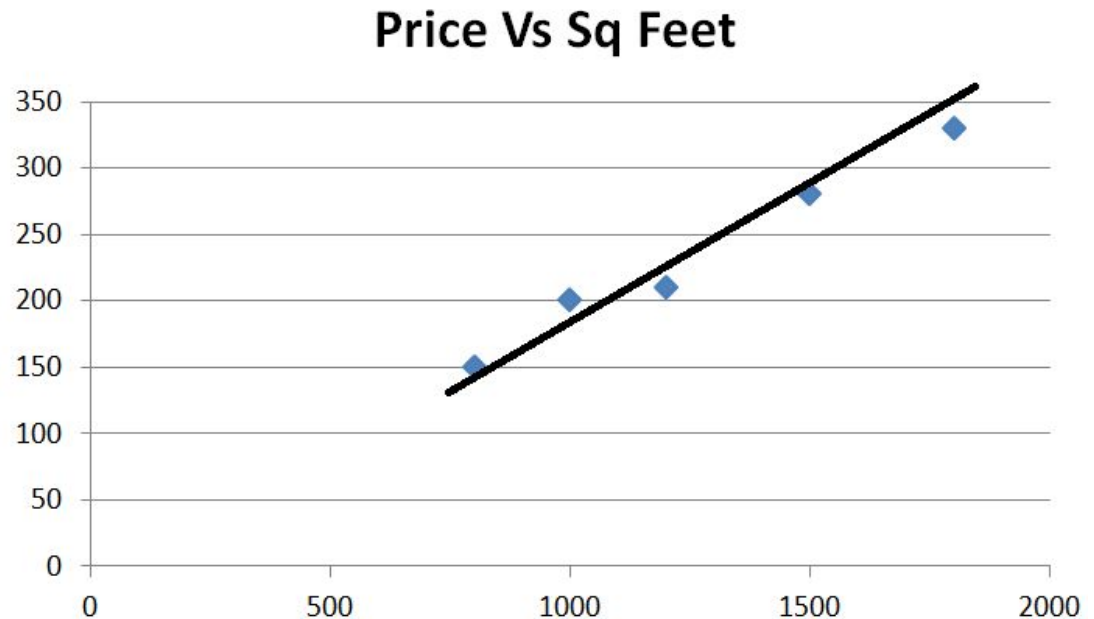
- **Problem:** Predict the selling price of a house (Y) based on its size in square feet (X).
- **Toy Training Data:**

Square Feet (X)	Price in \$1000s (Y)
800	150
1000	200
1200	210
1500	280
1800	330

- **Input Feature (X):** Square Feet (Continuous)
- **Output (Y):** Price (Continuous) -> **Prediction/Regression Problem**

Visualizing a Simple Predictor

- We want to find a *line* that best represents the trend in the data.



- **The Line: $\hat{f}(x) = \theta_0 + \theta_1 * x$**
 - θ_0 (Theta-Zero): The intercept.
 - θ_1 (Theta-One): The slope (how much price increases per sq. foot).

Mathematical Deep Dive (LR)

Let's derive the objective for our toy house price example.

- **Goal:** Minimize the **Sum of Squared Errors (SSE)** across all 'n' data points.
- **The Cost Function J(θ):**

$$J(\theta_0, \theta_1) = \sum_{i=1}^n (y^{(i)} - \hat{f}(x^{(i)}))^2$$

- Substitute the line equation

$$(\hat{f}(x) = \theta_0 + \theta_1 * x):$$

$$J(\theta_0, \theta_1) = \sum_{i=1}^n (y^{(i)} - (\theta_0 + \theta_1 x^{(i)}))^2$$

- $y^{(i)}$: The actual price of the i-th house in the data.
- $x^{(i)}$: The size of the i-th house.
- n : Total number of houses in the dataset.

The algorithm's job is to find θ_0 and θ_1 that make $J(\theta_0, \theta_1)$ as small as possible.

Stepping Through the Calculation

- Let's calculate the error for a candidate line: **Price = 20 + 0.15 * SqFt** ($\theta_0=20$, $\theta_1=0.15$).

x (SqFt)	y (Actual Price)	Predicted Price ($\hat{y}$) = 20 + 0.15*x	Error (y - $\hat{y}$)	Squared Error
800	150	20 + 0.15*800 = 20 + 120 = 140	150 - 140 = 10	100
1000	200	20 + 0.15*1000 = 20 + 150 = 170	200 - 170 = 30	900
1200	210	20 + 0.15*1200 = 20 + 180 = 200	210 - 200 = 10	100
1500	280	20 + 0.15*1500 = 20 + 225 = 245	280 - 245 = 35	1225
1800	330	20 + 0.15*1800 = 20 + 270 = 290	330 - 290 = 40	1600

- Total Cost J = 100 + 900 + 100 + 1225 + 1600 = 3925**
- The algorithm will try different θ_0 , θ_1 values (e.g., 10, 0.18) to get a cost lower than 3925.

Example Dataset: Student Performance

Student | Study Hours | Attendance | Assignments | Result

- S1 | 2 | 60 | 3 | Fail
- S2 | 5 | 75 | 6 | Pass
- S3 | 3 | 65 | 4 | Fail
- S4 | 7 | 90 | 8 | Pass
- S5 | 6 | 85 | 7 | Pass

Goal:

- $f(\text{hours, attendance, assignments}) \rightarrow \text{Result}$

Sumarized Components of Supervised Learning

1. Feature Space
2. Target Variable
3. Hypothesis Function
4. Loss Function

All supervised learning algorithms revolve around these elements.

Lets apply our learning to a Problem

- Traffic rule for minimum distance between cars
- Problem Formulation
 - Breaking Distance as a function of Speed Limit
- Data

Speed Limit (m/s)	stopping distance
10	5
20	20

Supervised Learning Problem

1. Feature Space
2. Target Variable
3. Hypothesis Function
4. Loss Function

Supervised Learning Problem

1. Feature Space

- 1. Speed Limit

2. Target Variable

- 1. Stopping Distance

3. Loss Function

- 1. Mean Squared Error

4. Hypothesis Function

- 1. Quadratic (why)

Loss Function

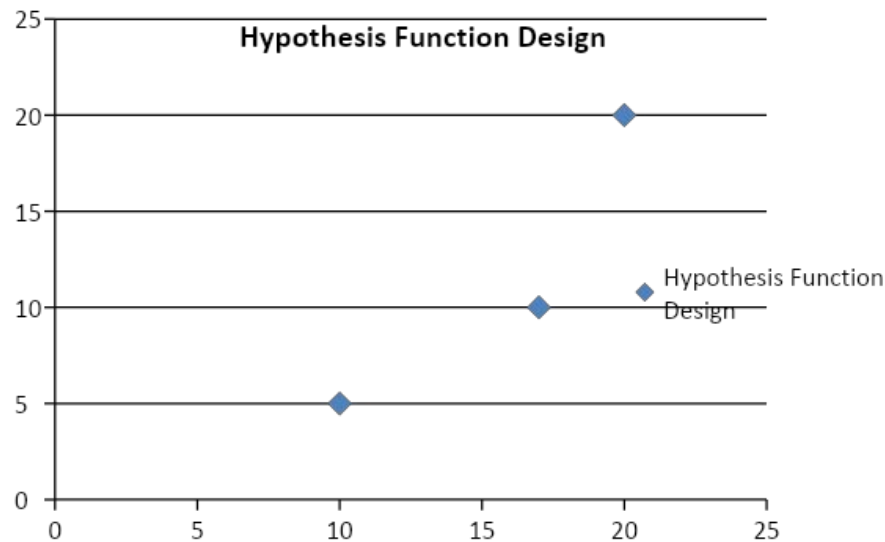
- We want to minimize the **Mean Squared Error (MSE)** for our data points:

$$J(\theta) = \frac{1}{2} \sum (h_{\theta}(x) - y)^2$$

- Point 1 : ($x_1 = 10$, $y_1 = 5$)
- Point 2 : ($x_2 = 20$, $y_2 = 20$)

Hypothesis Function

- Replace $h_{\theta}(x)$ with our hypothesis
- Our Hypothesis is of quadratic form with only One learnable (or tunable) parameter



$$\theta x^2$$

Substitute the Hypothesis

- Replace $h_{\theta}(x)$ with our hypothesis
- Our Hypothesis is of quadratic form (why ?)

$$\theta x^2$$

$$J(\theta) = \frac{1}{2} \sum (h_{\theta}(x) - y)^2$$

$$J(\theta) = \frac{1}{2} [(\theta(10)^2 - 5)^2 + (\theta(20)^2 - 20)^2]$$

$$J(\theta) = \frac{1}{2} [(100\theta - 5)^2 + (400\theta - 20)^2]$$

Find the Derivative (The Gradient)

- To find the minimum, we find where the slope of the loss curve is zero. We take the derivative of $J(\theta)$ with respect to θ

$$\frac{d}{d\theta} J(\theta) = 100(100\theta - 5) + 400(400\theta - 20)$$

Solve for θ

$$\frac{d}{d\theta} J(\theta) = 100(100\theta - 5) + 400(400\theta - 20)$$

- Expand and Solve

$$10,000\theta - 500 + 160,000\theta - 8,000 = 0$$

$$170,000\theta = 8,500$$

$$\theta = \frac{8,500}{170,000} = \mathbf{0.05}$$

Final Decision

- Original Problem Formulation
 - Traffic rule for minimum distance between cars
 - Problem Formulation
 - Breaking Distance as a function of Speed Limit
- The ideal Parameter is
 - $\theta = 0.05$
- Our Hypothesis function
 - $\Theta x^2 = 0.05 * x^2$

So Stopping Distance = $0.05 * (\text{speed-limit})^2$

Training and Testing Error

- Training Error

$$J(\theta) = \frac{1}{2} \sum (h_{\theta}(x) - y)^2$$

- $J(\theta) = \frac{1}{2} [(\theta * 10^2 - 5)^2 + (\theta * 20^2 - 20)^2]$
 $= \frac{1}{2} [(0.05 * 100 - 5)^2 + (0.05 * 400 - 20)^2]$
 $= \frac{1}{2} [(5 - 5)^2 + (20 - 20)^2]$
 $= 0$

- Testing Error
 - No data point left for testing

Let's Analyze this Traffic Rule model

- What was the training stage

$$dJ(\theta)/d\theta == 0$$

- We were able to solve this using Analytical approach because we had a very simple problem:
 - 1 Unknown + trainable parameter (θ) &
 - 1 equation : $dJ(\theta)/d\theta == 0$
- Why was our problem so simple (??)

Hypothesis Function

Hypothesis Function

- Our Hypothesis Function is simple because :
 - It is a quadratic function (?)
 - ***It had only 1 variable/tunable/learnable parameter(?)***
 - It had only 1 additive term θx^2
- Some other Hypothesis :
 - Generalized Polynomial upto n-degree
 - $a + bx + cx^2 + dx^3 + ex^4$ (for $n=4$)
 - Support Vector Machine
 - Perceptron
 - Single Layer Neural Net
 - Deep Neural Net

Traffic Rule Model Analysis

- How realistic is that In other data mining cases
- Lets take a complex hypothesis function :
 - Polynomial of degree 100 & 1 input feature space
 - $a_0 + a_1x + a_2x^2 + \dots + a_{100}x^{100}$
 - 101 tunable/tranable parameters
 - Input feature space of 10
 - Features = $x_1, x_2, , , , x_{10}$
 - Hypothesis Function becomes a matrix :
 - $a_{00} + a_{01}x_0 + a_{02}x_0^2 + \dots + a_{0100}x_0^{100}$
 - $a_{10} + a_{11}x_1 + a_{12}x_1^2 + \dots + a_{1100}x_1^{100}$
 -
 - $a_{1000} + a_{1001}x_{10} + a_{1002}x_{10}^2 + \dots + a_{100100}x_{10}^{100}$

Followed by some operation to map the 10 dimension output to a single target variable (assuming target variable is in 1 dim. feature space)

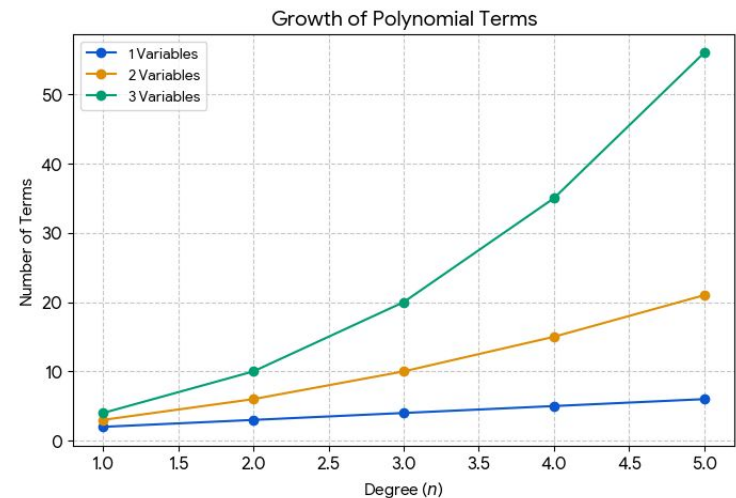
- What if the hypothesis function is not pure polynomial and a mixed polynomial

Variables (m)	Degree (n)	Max Terms	Example Expansion
1 (x)	2	3	$ax^2 + bx + c$
2 (x, y)	1	3	$ax + by + c$
2 (x, y)	2	6	$ax^2 + by^2 + cxy + dx + ey + f$
3 (x, y, z)	2	10	$ax^2 + by^2 + cz^2 + dxy + exz + fyz + gx + hy + iz + j$

Trainable params

$$(n+m)!/n!m!$$

- Where
 - n = degree of polynomial
 - m = input features space
- Analytical Approach isnt feasible !!!

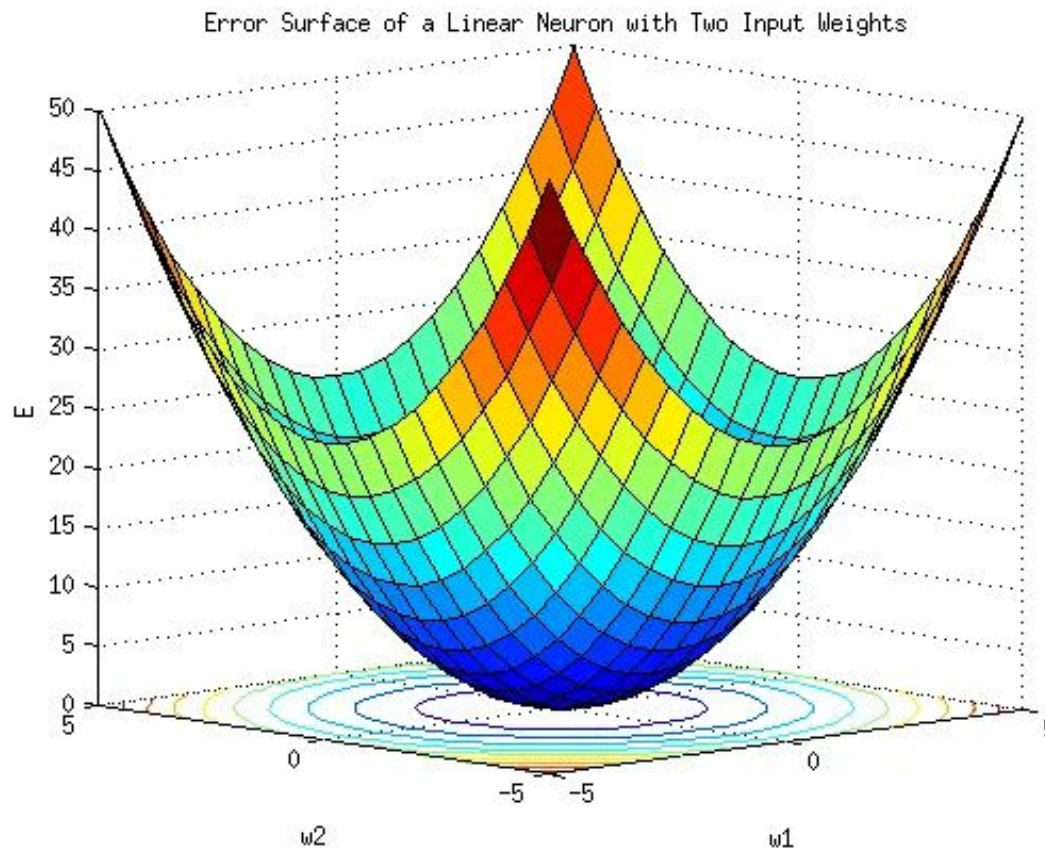


An iterative approach



Loss Minimization Via Downward hill approach

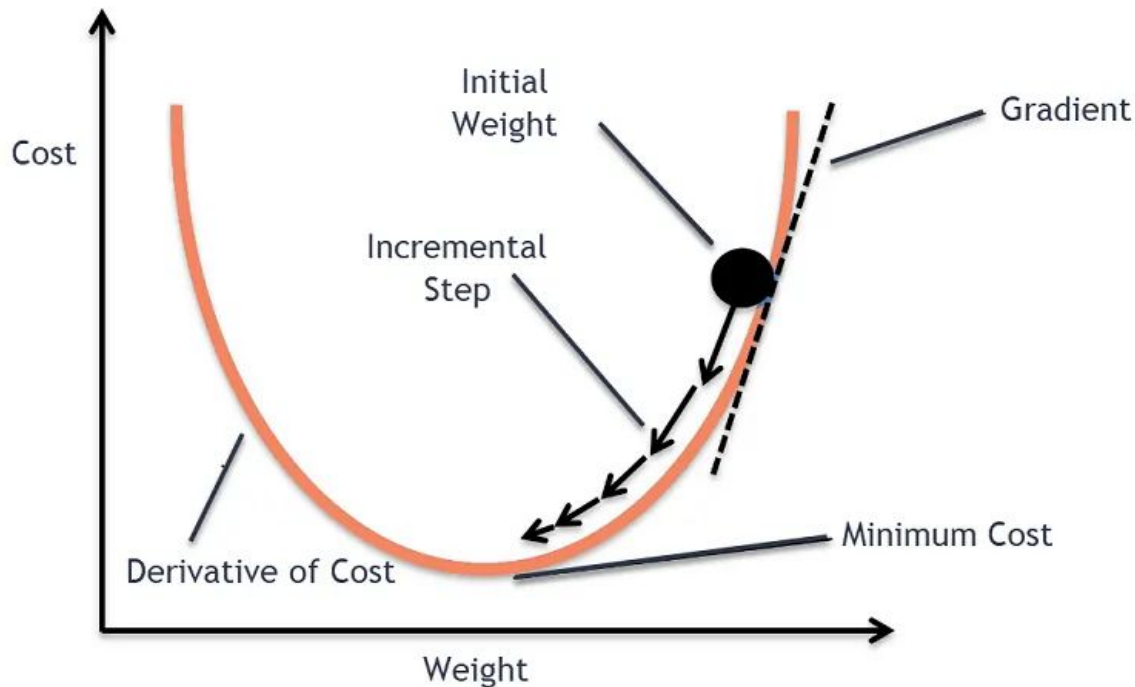
Take a step towards downward (opposite of upward) slope



Gradient Descent Algorithm

- The Algorithm
 - Compute the gradient
 - Make a step(α) in the -ve direction to the gradient.

$$\theta_{\text{new}} = \theta_{\text{old}} - \alpha \cdot \nabla J(\theta)$$



Another DM Approach

Lets try to solve exactly the same problem with another approach. Few salient points about teh new approach :

- Non Analytical ☐ No need to solve equations
- Not an exact solution, But good enough
- How good ☐ depends on compute power
 - Small Data, Faster Convergence ☐ Better accuracy
 - Big Data, Slower Convergence ☐ Low accuracy
- Incremental and Iterative
 - Stopping Criteria ☐ Model Convergence

The Setup

- Same Old Assumptions
 - Hypothesis $h(\theta) = \theta x^2$
 - Loss Function $J(\theta) = \frac{1}{2} \sum (h(\theta) - y)^2$
 - Summed over all data points
 - Data Points = (10,5) & (20,20)
- New Assumptions
 - Initial Guess $\theta = 0.1$
 - The ideal is .05, let's see how it varies
 - Learning Rate (α) = 0.000001
 - to prevent overshooting

Gradient Descent in Action

Lets see how our guess ($\theta=0.1$) performs

- Prediction : $h(\theta) = \theta x^2$
- Data 1 : $x=10, y=5$
 - Prediction $\hat{y} = 0.1 * 10^2 = 10$
 - Error = $10 - 5 = 5$
- Data 2 : $x=20, y=20$
 - Prediction $\hat{y} = 0.1 * 20^2 = 40$
 - Error = $40 - 20 = 20$

Predicted
Observed
Error

Gradient Descent in Action

- Compute the Slope

$$\frac{d}{d\theta} J(\theta) = 100(100\theta - 5) + 400(400\theta - 20)$$

- For Data1 = $100*(100*\theta - 5) = 100*(10 - 5) = 5$
- For Data2 = $400*(400*\theta - 20) = 400*20 = 8000$
- Average Gradient = **$8500/2 = 4250$**

Interpretation: Since the gradient is positive, the loss is currently increasing as θ increases. To reduce loss, we must decrease θ

Gradient Descent in Action

- Take the Step (aka update param θ)

$$\theta_{\text{new}} = \theta_{\text{old}} - \alpha \cdot \nabla J(\theta)$$

- $\Theta_{\text{new}} = 0.1 - 0.000001 * 4250$
- $\Theta_{\text{new}} = 0.1 - 0.004250$
- $\Theta_{\text{new}} = 0.09575$

Gradient Descent in Action

Result of Iteration 1

- Starting $\theta = 0.1$
- New $\theta = 0.09575$

Observation : ***The algorithm successfully nudged downwards from toward the ideal value of 0.05***

Question for the class

- Share results of iteration 2
- Share results of Iteration 1 with learning rate = 0.1
- Write code for parameter updation